PerfectEN - CHƯƠNG 1
NETWORK CONTROL PLANE

Bản tóm tắt để đọc nhanh và ôn thi
Nguồn: slide-en Chapter 5 (Network Layer: Control Plane)
Toàn bộ nội dung từ slide tiếng Anh (slide-vi không có)

1 Hai cách tổ chức control plane	2 Link state (Dijkstra)	3 Distance vector
4 Intra-AS: OSPF	5 Inter-AS: BGP	6 SDN
7 Network management	★ Ôn tập và câu hỏi	

Mục tiêu bài học

- Phân biệt control plane theo từng router với control plane tập trung (SDN).
- Chạy được thuật toán Dijkstra (link state) và Bellman-Ford (distance vector); so sánh LS với DV.
- Hiểu định tuyến theo AS: OSPF bên trong AS, BGP giữa các AS; kiến trúc SDN, OpenFlow; SNMP và NETCONF/YANG.

Chú thích: file này chỉ có nội dung từ slide-en nên không dùng khối tím. ICMP và traceroute của chapter này đã nằm trong Chương 7. Chỗ ghi **[cần kiểm tra lại]** là điểm slide chỉ có hình.

1 Hai cách tổ chức control plane

- **Per-router control** (truyền thống): thành phần thuật toán định tuyến chạy ở từng router và tương tác với nhau để tính forwarding table. Router là khối nguyên khối (phần cứng chuyển mạch + phần mềm độc quyền, ví dụ Cisco IOS) chạy IP, RIP, IS-IS, OSPF, BGP; các "middlebox" riêng cho firewall, load balancer, NAT.
- **Logically centralized control** (SDN): một controller từ xa tính và nạp forwarding table cho các router/switch. Quan tâm lại từ khoảng 2005.
- **Mục tiêu của routing protocol:** tìm đường (path) "tốt" - chi phí thấp nhất, nhanh nhất hoặc ít tắc nghẽn nhất - từ host gửi đến host nhận qua mạng các router. Mạng được trừu tượng hóa thành **đồ thị G = (N, E)**: N là tập router, E là tập link, $c_{a,b}$ là chi phí link trực tiếp a-b ($c_{a,b} = \infty$ nếu không nối trực tiếp). Chi phí do nhà vận hành định nghĩa: luôn bằng 1, tỉ lệ nghịch với băng thông, hoặc tỉ lệ nghịch với mức tắc nghẽn.

2 Link state: thuật toán Dijkstra

- Thuật toán **tập trung**: topo mạng và chi phí link được **mọi nút** biết nhờ **link-state broadcast**; tính đường đi chi phí nhỏ nhất từ một nút nguồn đến mọi nút khác, từ đó có forwarding table. Lặp: sau k vòng biết đường ngắn nhất tới k đích.

$D(v) = \min(D(v), D(w) + c_{w,v})$ Cập nhật cho mọi v kề w và chưa thuộc N'	**$N' = \{u\}$; $D(v) = c_{u,v}$ hoặc ∞** Khởi tạo: u chỉ biết chi phí tới láng giềng trực tiếp

Ký hiệu: **N'** tập nút đã xác định xong đường ngắn nhất; **D(v)** chi phí đường đi hiện tốt nhất từ nguồn tới v; **p(v)** nút đứng trước v trên đường đó; $c_{w,v}$ chi phí link w-v.

- **Các bước lặp:** (1) chọn w không thuộc N' sao cho D(w) nhỏ nhất; (2) thêm w vào N'; (3) cập nhật D(v) cho mọi v kề w và chưa thuộc N'; lặp đến khi mọi nút thuộc N'. Dựng cây đường đi ngắn nhất bằng cách lần theo các nút đứng trước p(v); khi bằng nhau (ties) có thể chọn tùy ý.
- **Độ phức tạp:** n nút, mỗi vòng kiểm tra mọi nút ngoài N' → n(n+1)/2 phép so sánh = **$O(n^2)$**; cài đặt tốt hơn có thể O(n log n). **Độ phức tạp thông điệp:** mỗi router broadcast link state tới n router → $O(n^2)$ tổng thể.
- **Dao động (oscillation):** khi chi phí link phụ thuộc lưu lượng, đường đi có thể dao động liên tục.

3 Distance vector: thuật toán Bellman-Ford

- **Phương trình Bellman-Ford** (quy hoạch động): $d_x(y) = \min_v \{ c_{x,v} + d_v(y) \}$, với min lấy trên mọi láng giềng v của x. Ví dụ slide: u có láng giềng x, v, w với c = 2, 1, 5 và biết $d_x(z) = 5$, $d_v(z) = 3$, $d_w(z) = 3$ thì $d_u(z) = \min\{2+5, 1+3, 5+3\} = \mathbf{4}$ (đi qua v).
- **Ý tưởng:** thỉnh thoảng mỗi nút gửi **vector khoảng cách** D_x của mình cho láng giềng; trong điều kiện bình thường, ước lượng $D_x(y)$ hội tụ về chi phí thật $d_x(y)$.
- **Tính chất:** lặp, bất đồng bộ (mỗi vòng do thay đổi chi phí link cục bộ hoặc nhận DV từ láng giềng); phân tán và tự dừng - chỉ thông báo khi DV của mình đổi; không nhận thông báo thì không làm gì.
- **Mỗi vòng mỗi nút:** nhận DV từ láng giềng → tính DV mới → gửi DV mới cho láng giềng.

Ví dụ tính toán trong slide: b nhận DV từ a, c, e

- $D_b(a) = \min\{c_{b,a}+D_a(a), c_{b,c}+D_c(a), c_{b,e}+D_e(a)\} = \min\{8, \infty, \infty\} = \mathbf{8}$; $D_b(c) = \mathbf{1}$; $D_b(d) = \min\{9, 2, \infty\} = \mathbf{2}$; $D_b(f) = \mathbf{2}$; $D_b(h) = \mathbf{2}$; $D_b(e) = \mathbf{1}$; $D_b(g) = D_b(i) = \infty$.
- Ví dụ c nhận DV từ b: $D_c(a) = 1 + 8 = 9$; $D_c(e) = 1 + 1 = 2$.

- **Thay đổi chi phí link:** *good news travels fast* - tin chi phí giảm lan nhanh (y phát hiện, báo cho z, z cập nhật, y thấy không đổi nên dừng). *Bad news travels slow* - **count-to-infinity**: khi chi phí link y-x tăng lên 60 trong khi z báo có đường tới x chi phí 5, y tính "6 qua z", z tính "7 qua y", y tính "8", ... cứ tăng dần (slide: xem giải pháp trong giáo trình).

Tiêu chí	Link State (LS)	Distance Vector (DV)
Thông điệp	n router: $O(n^2)$ thông điệp	Trao đổi giữa láng giềng; thời gian hội tụ thay đổi
Tốc độ hội tụ	Thuật toán $O(n^2)$; có thể dao động	Thay đổi; có thể có vòng lặp định tuyến, count-to-infinity
Độ bền khi router lỗi hoặc bị tấn công	Có thể quảng bá sai chi phí link; mỗi router chỉ tính bảng của mình	Có thể quảng bá sai chi phí đường đi ("tôi có đường rất rẻ tới mọi nơi") gây **black-holing**; bảng của mỗi router được dùng bởi các router khác nên lỗi lan truyền

4 Định tuyến trong AS: autonomous system và OSPF

- **Vì sao cần phân cấp:** hàng tỉ đích đến không thể ghi hết trong bảng, trao đổi bảng sẽ làm nghẽn link; mỗi nhà quản trị muốn tự quản lý định tuyến của mạng mình (Internet là mạng của các mạng).
- Gom router thành các **autonomous system (AS)** (hay domain). **Intra-AS**: định tuyến trong cùng một AS, mọi router trong AS chạy cùng một giao thức nội miền, các AS khác nhau có thể dùng giao thức khác nhau. **Gateway router** nằm ở rìa AS, có link sang AS khác và làm cả định tuyến liên miền (**inter-AS**).
- Forwarding table do cả hai: intra-AS quyết định mục cho đích trong AS; inter-AS cùng intra-AS quyết định mục cho đích bên ngoài (router phải biết đích nào đi qua AS nào và gateway nào).
- **Giao thức intra-AS phổ biến: RIP** [RFC 1723] - DV cổ điển, trao đổi DV mỗi 30 s, nay ít dùng; **EIGRP** - dựa trên DV, trước đây độc quyền Cisco (mở từ 2013); **OSPF** [RFC 2328] - link state; **IS-IS** - chuẩn ISO, về cơ bản giống OSPF.

OSPF (Open Shortest Path First)

- "Open": công khai. Link state cổ điển: mỗi router **flood** các link-state advertisement (gửi thẳng trên IP, không qua TCP/UDP) tới mọi router trong AS; có nhiều metric (băng thông, độ trễ).
- Mỗi router có đầy đủ topo và dùng **Dijkstra** để tính forwarding table.
- **OSPF phân cấp 2 mức:** local area và backbone; LSA chỉ flood trong area hoặc backbone; mỗi nút biết chi tiết topo area của mình và chỉ biết hướng đi tới đích ngoài area.

5 Định tuyến giữa các AS: BGP

- **BGP** (Border Gateway Protocol) là giao thức liên miền de facto, "chất keo gắn Internet": cho phép một mạng quảng bá "tôi ở đây và tôi tới được những đích này, bằng đường này".
- Mỗi AS: **eBGP** lấy thông tin tới được (reachability) từ AS láng giềng; **iBGP** truyền thông tin đó tới mọi router trong AS; rồi chọn đường "tốt" dựa trên reachability và **chính sách** (policy).
- **BGP session:** hai router BGP (peer) trao đổi thông điệp qua kết nối **TCP** bán thường trực, quảng bá đường tới các **prefix** đích (BGP là giao thức **path vector**). Khi gateway 3a của AS3 quảng bá "AS3, X" cho AS2, AS3 hứa chuyển datagram về phía X.
- **Route = prefix + thuộc tính.** Hai thuộc tính quan trọng: **AS-PATH** (danh sách các AS mà quảng bá đã đi qua) và **NEXT-HOP** (router nội bộ AS dẫn tới AS kế tiếp). Gateway dùng **import policy** để chấp nhận hay từ chối đường (ví dụ không bao giờ đi qua AS Y) và policy quyết định có quảng bá tiếp cho láng giềng không.
- **4 loại thông điệp BGP: OPEN** (mở kết nối TCP và xác thực peer), **UPDATE** (quảng bá đường mới hoặc thu hồi đường cũ), **KEEPALIVE** (giữ kết nối; cũng ACK cho OPEN), **NOTIFICATION** (báo lỗi, cũng dùng để đóng kết nối).

Chọn đường BGP (theo thứ tự)	Ý nghĩa
1. Local preference	Giá trị do chính sách của AS đặt
2. AS-PATH ngắn nhất	Ít AS nhất
3. NEXT-HOP gần nhất	**Hot potato routing**: chọn gateway có chi phí nội miền (theo OSPF) thấp nhất, bất kể số AS hop phía sau

Chọn đường BGP (theo thứ tự)	Ý nghĩa
4. Tiêu chí bổ sung	Slide không liệt kê chi tiết

- **Vì sao khác nhau giữa intra-AS và inter-AS:** về **policy** - inter-AS cần kiểm soát lưu lượng đi qua mạng mình, intra-AS chỉ một nhà quản trị nên policy ít quan trọng; về **scale** - phân cấp giảm kích thước bảng và lưu lượng cập nhật; về **performance** - intra-AS tập trung hiệu năng, inter-AS thì policy quan trọng hơn hiệu năng.
- **Chính sách qua quảng bá:** ISP chỉ muốn chuyển lưu lượng từ/đến khách hàng của mình, không làm trung chuyển giữa các ISP khác. Ví dụ: A quảng bá đường Aw cho B và C; B không quảng bá BAw cho C vì B không có doanh thu khi chuyển CBAw; C sẽ đi C-A-w. Mạng khách x kết nối hai nhà cung cấp (dual-homed) sẽ không quảng bá cho B đường tới C để không bị dùng làm đường trung chuyển B-x-C.

6 SDN - Software Defined Networking

- **Vì sao tập trung logic:** quản lý mạng dễ hơn (tránh cấu hình sai, linh hoạt hơn về luồng lưu lượng); forwarding dựa trên bảng (OpenFlow API) cho phép "lập trình" router; tính bảng tập trung rồi phân phối dễ hơn là chạy thuật toán phân tán trên mọi router; cài đặt mở thúc đẩy đổi mới. Phép so sánh: từ máy tính lớn khép kín (phần cứng + OS + ứng dụng chuyên biệt, đổi mới chậm) sang PC mở (giao diện mở, đổi mới nhanh).
- **Traffic engineering khó với định tuyến truyền thống:** muốn lưu lượng u→z đi u-v-w-z thay vì u-x-y-z thì chỉ có thể đổi link weight; muốn chia tải trên hai đường thì không làm được; muốn w định tuyến lưu lượng "xanh" và "đỏ" khác nhau cũng không được (forwarding theo đích). SDN và generalized forwarding làm được mọi thứ này.

Thành phần SDN	Vai trò
Data-plane switch	Switch nhanh, đơn giản, phần cứng thông dụng, thực hiện generalized forwarding; flow table được controller tính và nạp; có API điều khiển (ví dụ OpenFlow) xác định cái gì điều khiển được
SDN controller (network OS)	Giữ trạng thái mạng; giao tiếp lên ứng dụng qua **northbound API**, xuống switch qua **southbound API**; cài đặt như hệ phân tán để mở rộng và chịu lỗi. Gồm: giao tiếp với thiết bị, quản lý trạng thái toàn mạng (cơ sở dữ liệu phân tán), giao diện trừu tượng cho ứng dụng
Network-control apps	"Bộ não": routing, access control, load balance...; dùng dịch vụ của controller; có thể do bên thứ ba cung cấp

- **OpenFlow**: giao thức giữa controller và switch, dùng **TCP**, mã hóa tùy chọn; 3 loại thông điệp: controller-to-switch, bất đồng bộ (switch-to-controller), đối xứng. Khác với OpenFlow API (dùng để chỉ định hành động forwarding tổng quát).
- **Controller → switch:** features (hỏi tính năng), configure, **modify-state** (thêm/xóa/sửa flow entry), **packet-out** (gửi một gói ra cổng chỉ định). **Switch → controller: packet-in** (chuyển gói lên controller), **flow-removed** (entry bị xóa), **port status** (cổng thay đổi). Thực tế người vận hành dùng trừu tượng bậc cao ở controller, không gõ trực tiếp OpenFlow.
- **Ví dụ tương tác:** khi link state thay đổi, controller chạy Dijkstra, tính lại flow table rồi nạp xuống các switch (slide 81-82 là hình **[cần kiểm tra lại]**).
- **Controller thực tế: OpenDaylight (ODL)** có Service Abstraction Layer (SAL) nối ứng dụng trong và ngoài; **ONOS** tách ứng dụng khỏi controller, có intent framework (mô tả "cái gì" thay vì "như thế nào") và nhấn mạnh lõi phân tán.
- **Thách thức:** làm control plane bền vững, tin cậy, bảo mật, mở rộng ở quy mô Internet (vượt một AS); SDN rất quan trọng trong mạng di động 5G.

7 Quản lý mạng: SNMP và NETCONF/YANG

- Thành phần: **managing server** (ứng dụng, có người quản lý), **managed device** (thiết bị cấu hình được) và **data** (cấu hình, dữ liệu vận hành, thống kê).
- **Ba cách quản lý: CLI** (gõ lệnh hoặc script trực tiếp tới từng thiết bị, ví dụ qua ssh); **SNMP/MIB** (truy vấn/đặt dữ liệu thiết bị bằng SNMP); **NETCONF/YANG** (trừu tượng hơn, toàn mạng, nhấn mạnh cấu hình nhiều thiết bị).
- **SNMP:** truyền thông tin MIB và lệnh theo mode request/response (và trap); PDU gồm loại thông điệp (0-3), trạng thái, các biến cần get/set. **MIB** (Management Information Base): dữ liệu vận hành (và một số cấu hình) của thiết bị gom thành MIB module, định nghĩa bằng **SMI**; có hơn 400 MIB module trong RFC, nhiều MIB riêng của hãng. Chi tiết loại thông điệp SNMP nằm trong hình slide 95 **[cần kiểm tra lại]**.
- **NETCONF:** chủ động quản lý/cấu hình thiết bị toàn mạng; hoạt động giữa managing server và thiết bị; thao tác: lấy, đặt, sửa, kích hoạt cấu hình; **atomic-commit** trên nhiều thiết bị; truy vấn dữ liệu vận hành và thống kê; đăng ký nhận thông báo. Mô hình **RPC**, thông điệp mã hóa **XML**, chạy trên giao vận bảo mật tin cậy (ví dụ TLS).
- **Các thao tác NETCONF chọn lọc:** <get-config> (lấy cấu hình), <get> (lấy cấu hình và dữ liệu vận hành), <edit-config> (đổi cấu hình, trả <ok> hoặc <rpc-error> có rollback), <lock>/<unlock> (khóa datastore, chặn lệnh

từ NETCONF/SNMP/CLI khác), <create-subscription> và <notification> (nhận thông báo sự kiện). Ví dụ slide: đổi MTU của interface Ethernet 0/0 thành 1500.

- **YANG:** ngôn ngữ mô hình hóa dữ liệu, mô tả cấu trúc, cú pháp, ngữ nghĩa của dữ liệu NETCONF; có kiểu dữ liệu dựng sẵn như SMI; có thể sinh tài liệu XML mô tả thiết bị; biểu diễn được ràng buộc giữa các dữ liệu để đảm bảo cấu hình hợp lệ và nhất quán.

★ Ôn tập nhanh

Cần thuộc

Dijkstra	Bellman-Ford	BGP	OpenFlow
$D(v) = \min(D(v), D(w) + c_{w,v})$, $O(n^2)$	$d_x(y) = \min_v\{c_{x,v} + d_v(y)\}$	eBGP + iBGP; path vector; chạy trên TCP	chạy trên TCP; packet-in / packet-out

Lỗi thường gặp

Dễ nhầm	Cần nhớ
LS và DV	LS: mọi nút biết cả topo, chạy Dijkstra. DV: chỉ biết láng giềng, trao đổi vector, Bellman-Ford
OSPF và RIP	OSPF: link state, flood LSA trên IP. RIP: distance vector, trao đổi mỗi 30 s
eBGP và iBGP	eBGP giữa các AS; iBGP trong cùng AS
Hot potato	Chọn gateway có chi phí nội miền nhỏ nhất, không quan tâm số AS phía sau
Northbound và southbound	Northbound: controller ↔ ứng dụng; southbound: controller ↔ switch
OpenFlow message vs OpenFlow API	Message: giao thức controller-switch; API: chỉ định hành động forwarding

Câu hỏi ôn tập (che cột đáp án rồi tự trả lời)

Câu hỏi	Đáp án
1. Hai cách tổ chức control plane?	Per-router (truyền thống) và logically centralized (SDN)
2. Độ phức tạp của Dijkstra?	$O(n^2)$ (có thể $O(n \log n)$ với cài đặt tốt hơn)
3. Tính $d_u(z)$ nếu $c = 2, 1, 5$ và $d_x(z) = 5$, $d_v(z) = 3$, $d_w(z) = 3$.	$\min\{2+5, 1+3, 5+3\} = 4$
4. Count-to-infinity xảy ra ở thuật toán nào, khi nào?	Distance vector, khi chi phí link tăng ("bad news travels slow")
5. OSPF dùng thuật toán và giao vận gì?	Dijkstra; gửi LSA trực tiếp trên IP, không qua TCP/UDP
6. Gateway router là gì?	Router ở rìa AS, nối sang AS khác và làm cả intra-AS lẫn inter-AS
7. Kể 4 thông điệp BGP.	OPEN, UPDATE, KEEPALIVE, NOTIFICATION
8. Hai thuộc tính BGP quan trọng?	AS-PATH và NEXT-HOP
9. Vì sao B không quảng bá BAw cho C?	B không có doanh thu khi chuyển lưu lượng giữa hai ISP khác (policy)
10. NETCONF dùng mã hóa nào và mô hình gì?	XML, mô hình RPC; dữ liệu mô tả bằng YANG

Thực hành gợi ý

- Tự chạy Dijkstra bằng bảng (N', D(v), p(v)) trên đồ thị 6 nút u, v, w, x, y, z của slide 9-14 và dựng cây đường đi ngắn nhất.
- Tự tính lại các ví dụ DV của nút b (slide 33) rồi tính tiếp vector của nút e.